IEModule3Assignment

October 21, 2026

1 Assignment 3: Bidirectional LSTM CRF Model

This week, you learned about the bidirectional LSTM and CRF models. In this assignment, you will see how we can implement these models in python.

[1]:
```
import json
import torch
import torch.nn as nn
from torchcrf import CRF
from torch.utils.data import Dataset, DataLoader
from sklearn.model_selection import train_test_split
import random
import numpy as np

# Load the dataset
with open("assets/list_of_real_usa_addresses.json", "r") as file:
    addresses_json = json.load(file)

# Define constants
PAD_TOKEN = "<PAD>"
UNK_TOKEN = "<UNK>"
```

2 Task 1: Set up data

In this task, you will be responsible for formatting the dataset into the arrays X and y, where X is a list of addresses split up into tokens, and y is a list containing the corresponding labels.

Ex. For the following data:

`[{ "address": "123 Main St", "city": "Springfield", "state": "IL", "zip": "62704" }, etc.]`

The function should create X and y like so:

`X = [["123", "Main", "St", "Springfield", "IL", "62704"], etc.]`

`y = [["address", "address", "address", "city", "state", "zip"], etc.]`

[2]:
```
task_id = '1'
```

```
## Utility Code will go here
```

[3]:
```
def task_1_solution():
    X = []
    y = []

    # YOUR CODE HERE
    #raise NotImplementedError()
    for address in addresses_json:
        Xi, yi = [], []
        for label, text in address.items():
            tokens = text.split()
            labels = [label for _ in tokens]
            Xi += tokens
            yi += labels
        X.append(Xi)
        y.append(yi)

    return X, y
```

[4]:
```
# Use this cell to explore your solution.

task_1_solution()
```

[4]:
```
([['777', 'Brockton', 'Avenue', 'Abington', 'MA', '2351'],
  ['30', 'Memorial', 'Drive', 'Avon', 'MA', '2322'],
  ['250', 'Hartford', 'Avenue', 'Bellingham', 'MA', '2019'],
  ['700', 'Oak', 'Street', 'Brockton', 'MA', '2301'],
  ['66-4', 'Parkhurst', 'Rd', 'Chelmsford', 'MA', '1824'],
  ['591', 'Memorial', 'Dr', 'Chicopee', 'MA', '1020'],
  ['55', 'Brooksby', 'Village', 'Way', 'Danvers', 'MA', '1923'],
  ['137', 'Teaticket', 'Hwy', 'East', 'Falmouth', 'MA', '2536'],
  ['42', 'Fairhaven', 'Commons', 'Way', 'Fairhaven', 'MA', '2719'],
  ['374', 'William', 'S', 'Canning', 'Blvd', 'Fall', 'River', 'MA', '2721'],
  ['121', 'Worcester', 'Rd', 'Framingham', 'MA', '1701'],
  ['677', 'Timpany', 'Blvd', 'Gardner', 'MA', '1440'],
  ['337', 'Russell', 'St', 'Hadley', 'MA', '1035'],
  ['295', 'Plymouth', 'Street', 'Halifax', 'MA', '2338'],
  ['1775', 'Washington', 'St', 'Hanover', 'MA', '2339'],
  ['280', 'Washington', 'Street', 'Hudson', 'MA', '1749'],
  ['20', 'Soojian', 'Dr', 'Leicester', 'MA', '1524'],
  ['11', 'Jungle', 'Road', 'Leominster', 'MA', '1453'],
  ['301', 'Massachusetts', 'Ave', 'Lunenburg', 'MA', '1462'],
  ['780', 'Lynnway', 'Lynn', 'MA', '1905'],
  ['70', 'Pleasant', 'Valley', 'Street', 'Methuen', 'MA', '1844'],
  ['830', 'Curran', 'Memorial', 'Hwy', 'North', 'Adams', 'MA', '1247'],
  ['1470', 'S', 'Washington', 'St', 'North', 'Attleboro', 'MA', '2760'],
```

```
['506', 'State', 'Road', 'North', 'Dartmouth', 'MA', '2747'],
['742', 'Main', 'Street', 'North', 'Oxford', 'MA', '1537'],
['72', 'Main', 'St', 'North', 'Reading', 'MA', '1864'],
['200', 'Otis', 'Street', 'Northborough', 'MA', '1532'],
['180', 'North', 'King', 'Street', 'Northhampton', 'MA', '1060'],
['555', 'East', 'Main', 'St', 'Orange', 'MA', '1364'],
['555', 'Hubbard', 'Ave-Suite', '12', 'Pittsfield', 'MA', '1201'],
['300', 'Colony', 'Place', 'Plymouth', 'MA', '2360'],
['301', 'Falls', 'Blvd', 'Quincy', 'MA', '2169'],
['36', 'Paramount', 'Drive', 'Raynham', 'MA', '2767'],
['450', 'Highland', 'Ave', 'Salem', 'MA', '1970'],
['1180', 'Fall', 'River', 'Avenue', 'Seekonk', 'MA', '2771'],
['1105', 'Boston', 'Road', 'Springfield', 'MA', '1119'],
['100', 'Charlton', 'Road', 'Sturbridge', 'MA', '1566'],
['262', 'Swansea', 'Mall', 'Dr', 'Swansea', 'MA', '2777'],
['333', 'Main', 'Street', 'Tewksbury', 'MA', '1876'],
['550', 'Providence', 'Hwy', 'Walpole', 'MA', '2081'],
['352', 'Palmer', 'Road', 'Ware', 'MA', '1082'],
['3005', 'Cranberry', 'Hwy', 'Rt', '6', '28', 'Wareham', 'MA', '2538'],
['250', 'Rt', '59', 'Airmont', 'NY', '10901'],
['141', 'Washington', 'Ave', 'Extension', 'Albany', 'NY', '12205'],
['13858', 'Rt', '31', 'W', 'Albion', 'NY', '14411'],
['2055', 'Niagara', 'Falls', 'Blvd', 'Amherst', 'NY', '14228'],
['101', 'Sanford', 'Farm', 'Shpg', 'Center', 'Amsterdam', 'NY', '12010'],
['297', 'Grant', 'Avenue', 'Auburn', 'NY', '13021'],
['4133', 'Veterans', 'Memorial', 'Drive', 'Batavia', 'NY', '14020'],
['6265', 'Brockport', 'Spencerport', 'Rd', 'Brockport', 'NY', '14420'],
['5399', 'W', 'Genesse', 'St', 'Camillus', 'NY', '13031'],
['3191', 'County', 'rd', '10', 'Canandaigua', 'NY', '14424'],
['30', 'Catskill', 'Catskill', 'NY', '12414'],
['161', 'Centereach', 'Mall', 'Centereach', 'NY', '11720'],
['3018', 'East', 'Ave', 'Central', 'Square', 'NY', '13036'],
['100', 'Thruway', 'Plaza', 'Cheektowaga', 'NY', '14225'],
['8064', 'Brewerton', 'Rd', 'Cicero', 'NY', '13039'],
['5033', 'Transit', 'Road', 'Clarence', 'NY', '14031'],
['3949', 'Route', '31', 'Clay', 'NY', '13041'],
['139', 'Merchant', 'Place', 'Cobleskill', 'NY', '12043'],
['85', 'Crooked', 'Hill', 'Road', 'Commack', 'NY', '11725'],
['872', 'Route', '13', 'Cortlandville', 'NY', '13045'],
['279', 'Troy', 'Road', 'East', 'Greenbush', 'NY', '12061'],
['2465', 'Hempstead', 'Turnpike', 'East', 'Meadow', 'NY', '11554'],
['6438', 'Basile', 'Rowe', 'East', 'Syracuse', 'NY', '13057'],
['25737', 'US', 'Rt', '11', 'Evans', 'Mills', 'NY', '13637'],
['901', 'Route', '110', 'Farmingdale', 'NY', '11735'],
['2400', 'Route', '9', 'Fishkill', 'NY', '12524'],
['10401', 'Bennett', 'Road', 'Fredonia', 'NY', '14063'],
['1818', 'State', 'Route', '3', 'Fulton', 'NY', '13069'],
```

```
['4300', 'Lakeville', 'Road', 'Geneseo', 'NY', '14454'],
['990', 'Route', '5', '20', 'Geneva', 'NY', '14456'],
['311', 'RT', '9W', 'Glenmont', 'NY', '12077'],
['200', 'Dutch', 'Meadows', 'Ln', 'Glenville', 'NY', '12302'],
['100', 'Elm', 'Ridge', 'Center', 'Dr', 'Greece', 'NY', '14626'],
['1549', 'Rt', '9', 'Halfmoon', 'NY', '12065'],
['5360', 'Southwestern', 'Blvd', 'Hamburg', 'NY', '14075'],
['103', 'North', 'Caroline', 'St', 'Herkimer', 'NY', '13350'],
['1000', 'State', 'Route', '36', 'Hornell', 'NY', '14843'],
['1400', 'County', 'Rd', '64', 'Horseheads', 'NY', '14845'],
['135', 'Fairgrounds', 'Memorial', 'Pkwy', 'Ithaca', 'NY', '14850'],
['2', 'Gannett', 'Dr', 'Johnson', 'City', 'NY', '13790'],
['233', '5th', 'Ave', 'Ext', 'Johnstown', 'NY', '12095'],
['601', 'Frank', 'Stottile', 'Blvd', 'Kingston', 'NY', '12401'],
['350', 'E', 'Fairmount', 'Ave', 'Lakewood', 'NY', '14750'],
['4975', 'Transit', 'Rd', 'Lancaster', 'NY', '14086'],
['579', 'Troy-Schenectady', 'Road', 'Latham', 'NY', '12110'],
['5783', 'So', 'Transit', 'Road', 'Lockport', 'NY', '14094'],
['7155', 'State', 'Rt', '12', 'S', 'Lowville', 'NY', '13367'],
['425', 'Route', '31', 'Macedon', 'NY', '14502'],
['3222', 'State', 'Rt', '11', 'Malone', 'NY', '12953'],
['200', 'Sunrise', 'Mall', 'Massapequa', 'NY', '11758'],
['43', 'Stephenville', 'St', 'Massena', 'NY', '13662'],
['750', 'Middle', 'Country', 'Road', 'Middle', 'Island', 'NY', '11953'],
['470', 'Route', '211', 'East', 'Middletown', 'NY', '10940'],
['3133', 'E', 'Main', 'St', 'Mohegan', 'Lake', 'NY', '10547'],
['288', 'Larkin', 'Monroe', 'NY', '10950'],
['41', 'Anawana', 'Lake', 'Road', 'Monticello', 'NY', '12701'],
['4765', 'Commercial', 'Drive', 'New', 'Hartford', 'NY', '13413'],
['1201', 'Rt', '300', 'Newburgh', 'NY', '12550'],
['255', 'W', 'Main', 'St', 'Avon', 'CT', '6001'],
['120', 'Commercial', 'Parkway', 'Branford', 'CT', '6405'],
['1400', 'Farmington', 'Ave', 'Bristol', 'CT', '6010'],
['161', 'Berlin', 'Road', 'Cromwell', 'CT', '6416'],
['67', 'Newton', 'Rd', 'Danbury', 'CT', '6810'],
['656', 'New', 'Haven', 'Ave', 'Derby', 'CT', '6418'],
['69', 'Prospect', 'Hill', 'Road', 'East', 'Windsor', 'CT', '6088'],
['150', 'Gold', 'Star', 'Hwy', 'Groton', 'CT', '6340'],
['900', 'Boston', 'Post', 'Road', 'Guilford', 'CT', '6437'],
['2300', 'Dixwell', 'Ave', 'Hamden', 'CT', '6514'],
['495', 'Flatbush', 'Ave', 'Hartford', 'CT', '6106'],
['180', 'River', 'Rd', 'Lisbon', 'CT', '6351'],
['420', 'Buckland', 'Hills', 'Dr', 'Manchester', 'CT', '6040'],
['1365', 'Boston', 'Post', 'Road', 'Milford', 'CT', '6460'],
['1100', 'New', 'Haven', 'Road', 'Naugatuck', 'CT', '6770'],
['315', 'Foxon', 'Blvd', 'New', 'Haven', 'CT', '6513'],
['164', 'Danbury', 'Rd', 'New', 'Milford', 'CT', '6776'],
```

```
['3164', 'Berlin', 'Turnpike', 'Newington', 'CT', '6111'],
['474', 'Boston', 'Post', 'Road', 'North', 'Windham', 'CT', '6256'],
['650', 'Main', 'Ave', 'Norwalk', 'CT', '6851'],
['680', 'Connecticut', 'Avenue', 'Norwalk', 'CT', '6854'],
['220', 'Salem', 'Turnpike', 'Norwich', 'CT', '6360'],
['655', 'Boston', 'Post', 'Rd', 'Old', 'Saybrook', 'CT', '6475'],
['625', 'School', 'Street', 'Putnam', 'CT', '6260'],
['80', 'Town', 'Line', 'Rd', 'Rocky', 'Hill', 'CT', '6067'],
['465', 'Bridgeport', 'Avenue', 'Shelton', 'CT', '6484'],
['235', 'Queen', 'St', 'Southington', 'CT', '6489'],
['150', 'Barnum', 'Avenue', 'Cutoff', 'Stratford', 'CT', '6614'],
['970', 'Torringford', 'Street', 'Torrington', 'CT', '6790'],
['844', 'No', 'Colony', 'Road', 'Wallingford', 'CT', '6492'],
['910', 'Wolcott', 'St', 'Waterbury', 'CT', '6705'],
['155', 'Waterford', 'Parkway', 'No', 'Waterford', 'CT', '6385'],
['515', 'Sawmill', 'Road', 'West', 'Haven', 'CT', '6516'],
['2473', 'Hackworth', 'Road', 'Adamsville', 'AL', '35005'],
['630', 'Coonial', 'Promenade', 'Pkwy', 'Alabaster', 'AL', '35007'],
['2643', 'Hwy', '280', 'West', 'Alexander', 'City', 'AL', '35010'],
['540', 'West', 'Bypass', 'Andalusia', 'AL', '36420'],
['5560', 'Mcclellan', 'Blvd', 'Anniston', 'AL', '36206'],
['1450', 'No', 'Brindlee', 'Mtn', 'Pkwy', 'Arab', 'AL', '35016'],
['1011', 'US', 'Hwy', '72', 'East', 'Athens', 'AL', '35611'],
['973', 'Gilbert', 'Ferry', 'Road', 'Se', 'Attalla', 'AL', '35954'],
['1717', 'South', 'College', 'Street', 'Auburn', 'AL', '36830'],
['701', 'Mcmeans', 'Ave', 'Bay', 'Minette', 'AL', '36507'],
['750', 'Academy', 'Drive', 'Bessemer', 'AL', '35022'],
['312', 'Palisades', 'Blvd', 'Birmingham', 'AL', '35209'],
['1600', 'Montclair', 'Rd', 'Birmingham', 'AL', '35210'],
['5919', 'Trussville', 'Crossings', 'Pkwy', 'Birmingham', 'AL', '35235'],
['9248', 'Parkway', 'East', 'Birmingham', 'AL', '35206'],
['1972', 'Hwy', '431', 'Boaz', 'AL', '35957'],
['10675', 'Hwy', '5', 'Brent', 'AL', '35034'],
['2041', 'Douglas', 'Avenue', 'Brewton', 'AL', '36426'],
['5100', 'Hwy', '31', 'Calera', 'AL', '35040'],
['1916', 'Center', 'Point', 'Rd', 'Center', 'Point', 'AL', '35215'],
['1950', 'W', 'Main', 'St', 'Centre', 'AL', '35960'],
['16077', 'Highway', '280', 'Chelsea', 'AL', '35043'],
['1415', '7Th', 'Street', 'South', 'Clanton', 'AL', '35045'],
['626', 'Olive', 'Street', 'Sw', 'Cullman', 'AL', '35055'],
['27520', 'Hwy', '98', 'Daphne', 'AL', '36526'],
['2800', 'Spring', 'Avn', 'SW', 'Decatur', 'AL', '35603'],
['969', 'Us', 'Hwy', '80', 'West', 'Demopolis', 'AL', '36732'],
['3300', 'South', 'Oates', 'Street', 'Dothan', 'AL', '36301'],
['4310', 'Montgomery', 'Hwy', 'Dothan', 'AL', '36303'],
['600', 'Boll', 'Weevil', 'Circle', 'Enterprise', 'AL', '36330'],
['3176', 'South', 'Eufaula', 'Avenue', 'Eufaula', 'AL', '36027'],
```

```
['7100', 'Aaron', 'Aronov', 'Drive', 'Fairfield', 'AL', '35064'],
['10040', 'County', 'Road', '48', 'Fairhope', 'AL', '36533'],
['3186', 'Hwy', '171', 'North', 'Fayette', 'AL', '35555'],
['3100', 'Hough', 'Rd', 'Florence', 'AL', '35630'],
['2200', 'South', 'Mckenzie', 'St', 'Foley', 'AL', '36535'],
['2001', 'Glenn', 'Bldv', 'Sw', 'Fort', 'Payne', 'AL', '35968'],
['340', 'East', 'Meighan', 'Blvd', 'Gadsden', 'AL', '35903'],
['890', 'Odum', 'Road', 'Gardendale', 'AL', '35071'],
['1608', 'W', 'Magnolia', 'Ave', 'Geneva', 'AL', '36340'],
['501', 'Willow', 'Lane', 'Greenville', 'AL', '36037'],
['170', 'Fort', 'Morgan', 'Road', 'Gulf', 'Shores', 'AL', '36542'],
['11697', 'US', 'Hwy', '431', 'Guntersville', 'AL', '35976'],
['42417', 'Hwy', '195', 'Haleyville', 'AL', '35565'],
['1706', 'Military', 'Street', 'South', 'Hamilton', 'AL', '35570'],
['1201', 'Hwy', '31', 'NW', 'Hartselle', 'AL', '35640'],
['209', 'Lakeshore', 'Parkway', 'Homewood', 'AL', '35209'],
['2780', 'John', 'Hawkins', 'Pkwy', 'Hoover', 'AL', '35244'],
['5335', 'Hwy', '280', 'South', 'Hoover', 'AL', '35242'],
['1007', 'Red', 'Farmer', 'Drive', 'Hueytown', 'AL', '35023'],
['2900', 'S', 'Mem', 'PkwyDrake', 'Ave', 'Huntsville', 'AL', '35801'],
['11610', 'Memorial', 'Pkwy', 'South', 'Huntsville', 'AL', '35803'],
['2200', 'Sparkman', 'Drive', 'Huntsville', 'AL', '35810'],
['330', 'Sutton', 'Rd', 'Huntsville', 'AL', '35763'],
['6140A', 'Univ', 'Drive', 'Huntsville', 'AL', '35806'],
['4206', 'N', 'College', 'Ave', 'Jackson', 'AL', '36545'],
['1625', 'Pelham', 'South', 'Jacksonville', 'AL', '36265'],
['1801', 'Hwy', '78', 'East', 'Jasper', 'AL', '35501'],
['8551', 'Whitfield', 'Ave', 'Leeds', 'AL', '35094'],
['8650', 'Madison', 'Blvd', 'Madison', 'AL', '35758'],
['145', 'Kelley', 'Blvd', 'Millbrook', 'AL', '36054'],
['1970', 'S', 'University', 'Blvd', 'Mobile', 'AL', '36609'],
['6350', 'Cottage', 'Hill', 'Road', 'Mobile', 'AL', '36609'],
['101', 'South', 'Beltline', 'Highway', 'Mobile', 'AL', '36606'],
['2500', 'Dawes', 'Road', 'Mobile', 'AL', '36695'],
['5245', 'Rangeline', 'Service', 'Rd', 'Mobile', 'AL', '36619'],
['685', 'Schillinger', 'Rd', 'Mobile', 'AL', '36695'],
['3371', 'S', 'Alabama', 'Ave', 'Monroeville', 'AL', '36460'],
['10710', 'Chantilly', 'Pkwy', 'Montgomery', 'AL', '36117'],
['3801', 'Eastern', 'Blvd', 'Montgomery', 'AL', '36116'],
['6495', 'Atlanta', 'Hwy', 'Montgomery', 'AL', '36117'],
['851', 'Ann', 'St', 'Montgomery', 'AL', '36107'],
['15445', 'Highway', '24', 'Moulton', 'AL', '35650'],
['517', 'West', 'Avalon', 'Ave', 'Muscle', 'Shoals', 'AL', '35661'],
['5710', 'Mcfarland', 'Blvd', 'Northport', 'AL', '35476'],
['2453', '2Nd', 'Avenue', 'East', 'Oneonta', 'AL', '35121'],
['2900', 'Pepperrell', 'Pkwy', 'Opelika', 'AL', '36801'],
['92', 'Plaza', 'Lane', 'Oxford', 'AL', '36203'],
```

```
 ['1537', 'Hwy', '231', 'South', 'Ozark', 'AL', '36360'],
 ['2181', 'Pelham', 'Pkwy', 'Pelham', 'AL', '35124'],
 ['165', 'Vaughan', 'Ln', 'Pell', 'City', 'AL', '35125'],
 ['3700', 'Hwy', '280-431', 'N', 'Phenix', 'City', 'AL', '36867'],
 ['1903', 'Cobbs', 'Ford', 'Rd', 'Prattville', 'AL', '36066'],
 ['4180', 'Us', 'Hwy', '431', 'Roanoke', 'AL', '36274'],
 ['13675', 'Hwy', '43', 'Russellville', 'AL', '35653'],
 ['1095', 'Industrial', 'Pkwy', 'Saraland', 'AL', '36571'],
 ['24833', 'Johnt', 'Reidprkw', 'Scottsboro', 'AL', '35768'],
 ['1501', 'Hwy', '14', 'East', 'Selma', 'AL', '36703'],
 ['7855', 'Moffett', 'Rd', 'Semmes', 'AL', '36575'],
 ['150', 'Springville', 'Station', 'Blvd', 'Springville', 'AL', '35146'],
 ['690', 'Hwy', '78', 'Sumiton', 'AL', '35148'],
 ['41301', 'US', 'Hwy', '280', 'Sylacauga', 'AL', '35150'],
 ['214', 'Haynes', 'Street', 'Talladega', 'AL', '35160'],
 ['1300', 'Gilmer', 'Ave', 'Tallassee', 'AL', '36078'],
 ['34301', 'Hwy', '43', 'Thomasville', 'AL', '36784'],
 ['1420', 'Us', '231', 'South', 'Troy', 'AL', '36081'],
 ['1501', 'Skyland', 'Blvd', 'E', 'Tuscaloosa', 'AL', '35405'],
 ['3501', '20th', 'Av', 'Valley', 'AL', '36854'],
 ['1300', 'Montgomery', 'Highway', 'Vestavia', 'Hills', 'AL', '35216'],
 ['4538', 'Us', 'Hwy', '231', 'Wetumpka', 'AL', '36092'],
 ['2575', 'Us', 'Hwy', '43', 'Winfield', 'AL', '35594']],
[['address', 'address', 'address', 'city', 'state', 'zip'],
 ['address', 'address', 'address', 'city', 'state', 'zip'],
 ['address', 'address', 'address', 'city', 'state', 'zip'],
 ['address', 'address', 'address', 'city', 'state', 'zip'],
 ['address', 'address', 'address', 'city', 'state', 'zip'],
 ['address', 'address', 'address', 'city', 'state', 'zip'],
 ['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
 ['address', 'address', 'address', 'city', 'city', 'state', 'zip'],
 ['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
 ['address',
  'address',
  'address',
  'address',
  'address',
  'city',
  'city',
  'state',
  'zip'],
 ['address', 'address', 'address', 'city', 'state', 'zip'],
 ['address', 'address', 'address', 'city', 'state', 'zip'],
 ['address', 'address', 'address', 'city', 'state', 'zip'],
 ['address', 'address', 'address', 'city', 'state', 'zip'],
 ['address', 'address', 'address', 'city', 'state', 'zip'],
 ['address', 'address', 'address', 'city', 'state', 'zip'],
```

```
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address',
 'address',
 'address',
 'address',
 'address',
 'city',
 'state',
 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address',
 'address',
 'address',
 'address',
 'address',
 'city',
 'state',
 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
```

```
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address',
 'address',
 'address',
 'address',
 'address',
 'city',
 'state',
 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
```

```
['address',
 'address',
 'address',
 'address',
 'address',
 'city',
 'state',
 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
```

```
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address',
 'address',
 'address',
 'address',
 'address',
 'city',
 'state',
 'zip'],
['address',
 'address',
 'address',
 'address',
 'address',
 'city',
 'state',
 'zip'],
['address',
 'address',
 'address',
 'address',
 'address',
 'city',
 'state',
 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
```

```
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address',
 'address',
 'address',
 'address',
 'address',
 'city',
 'state',
 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address',
 'address',
 'address',
 'address',
 'address',
 'city',
 'state',
 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
```

```
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip']])
```

[5]:
```
print(f"Task {task_id} - AG tests")

stu_X, stu_y = task_1_solution()

print(f"Task {task_id} - your X:\n{stu_X[:2]}")  # Display a subset of the data␣
 ↪for clarity
print(f"Task {task_id} - your y:\n{stu_y[:2]}")  # Display a subset of the data␣
 ↪for clarity

assert isinstance(stu_X, list), f"Task {task_id}: The returned `X` should be a␣
 ↪list."
assert isinstance(stu_y, list), f"Task {task_id}: The returned `y` should be a␣
 ↪list."

assert len(stu_X) == len(
    stu_y
), f"Task {task_id}: The lengths of `X` and `y` should match. Found␣
 ↪len(X)={len(stu_X)} and len(y)={len(stu_y)}."

assert all(
    isinstance(seq, list) and all(isinstance(word, str) for word in seq)
    for seq in stu_X
), f"Task {task_id}: Each element of `X` should be a list of strings."

assert all(
    isinstance(seq, list) and all(isinstance(label, str) for label in seq)
    for seq in stu_y
), f"Task {task_id}: Each element of `y` should be a list of strings."
```

```
Task 1 - AG tests
Task 1 - your X:
[['777', 'Brockton', 'Avenue', 'Abington', 'MA', '2351'], ['30', 'Memorial',
'Drive', 'Avon', 'MA', '2322']]
Task 1 - your y:
[['address', 'address', 'address', 'city', 'state', 'zip'], ['address',
'address', 'address', 'city', 'state', 'zip']]
```

3 Task 2: Create encodings for data:

To move further, we need to create a vocabulary and a label-to-index mapping strings to integer represntations to prepare the data for further processing.

The label-to-index mapping has been created for you, and the first elements of the address token vocabulary have been set for you.

In the function below, you need to populate `vocab` with the rest of the tokens seen in the X array from task_1.

For example, for an X like:

```
[["123", "Main", "St", "Springfield", "IL", "62704"], ["124", "Main", "St",
"Springfield", "IL", "62705"], etc.]
```

Your vocab would look like:

```
{     "<PAD>": 0,     "<UNK>": 1,     "123": 2,     "Main": 3,     "St": 4,
"Springfield": 5,     "IL": 6,     "62704": 7,     "124" : 8,     "62705" : 9,
etc. }
```

The order for assigning indices in vocab should be in order of appearance in X.

[ ]:
```
task_id = '2'
```

[6]:
```
def task_2_solution():
    X, _ = task_1_solution()

    vocab = {PAD_TOKEN: 0, UNK_TOKEN: 1}
    state_to_idx = {PAD_TOKEN: 0, "address": 1, "city": 2, "state": 3, "zip": 4}
    # YOUR CODE HERE
    #raise NotImplementedError()
    id = 2
    for x in X:
        for w in x:
            if not w in vocab:
                vocab[w] = id
                id += 1

    return vocab, state_to_idx
```

[7]:
```
# Use this cell to explore your solution.

task_2_solution()
```

[7]:
```
({'<PAD>': 0,
  '<UNK>': 1,
  '777': 2,
  'Brockton': 3,
  'Avenue': 4,
  'Abington': 5,
  'MA': 6,
  '2351': 7,
  '30': 8,
  'Memorial': 9,
  'Drive': 10,
  'Avon': 11,
  '2322': 12,
  '250': 13,
```

```
'Hartford': 14,
'Bellingham': 15,
'2019': 16,
'700': 17,
'Oak': 18,
'Street': 19,
'2301': 20,
'66-4': 21,
'Parkhurst': 22,
'Rd': 23,
'Chelmsford': 24,
'1824': 25,
'591': 26,
'Dr': 27,
'Chicopee': 28,
'1020': 29,
'55': 30,
'Brooksby': 31,
'Village': 32,
'Way': 33,
'Danvers': 34,
'1923': 35,
'137': 36,
'Teaticket': 37,
'Hwy': 38,
'East': 39,
'Falmouth': 40,
'2536': 41,
'42': 42,
'Fairhaven': 43,
'Commons': 44,
'2719': 45,
'374': 46,
'William': 47,
'S': 48,
'Canning': 49,
'Blvd': 50,
'Fall': 51,
'River': 52,
'2721': 53,
'121': 54,
'Worcester': 55,
'Framingham': 56,
'1701': 57,
'677': 58,
'Timpany': 59,
'Gardner': 60,
```

```
'1440': 61,
'337': 62,
'Russell': 63,
'St': 64,
'Hadley': 65,
'1035': 66,
'295': 67,
'Plymouth': 68,
'Halifax': 69,
'2338': 70,
'1775': 71,
'Washington': 72,
'Hanover': 73,
'2339': 74,
'280': 75,
'Hudson': 76,
'1749': 77,
'20': 78,
'Soojian': 79,
'Leicester': 80,
'1524': 81,
'11': 82,
'Jungle': 83,
'Road': 84,
'Leominster': 85,
'1453': 86,
'301': 87,
'Massachusetts': 88,
'Ave': 89,
'Lunenburg': 90,
'1462': 91,
'780': 92,
'Lynnway': 93,
'Lynn': 94,
'1905': 95,
'70': 96,
'Pleasant': 97,
'Valley': 98,
'Methuen': 99,
'1844': 100,
'830': 101,
'Curran': 102,
'North': 103,
'Adams': 104,
'1247': 105,
'1470': 106,
'Attleboro': 107,
```

```
'2760': 108,
'506': 109,
'State': 110,
'Dartmouth': 111,
'2747': 112,
'742': 113,
'Main': 114,
'Oxford': 115,
'1537': 116,
'72': 117,
'Reading': 118,
'1864': 119,
'200': 120,
'Otis': 121,
'Northborough': 122,
'1532': 123,
'180': 124,
'King': 125,
'Northhampton': 126,
'1060': 127,
'555': 128,
'Orange': 129,
'1364': 130,
'Hubbard': 131,
'Ave-Suite': 132,
'12': 133,
'Pittsfield': 134,
'1201': 135,
'300': 136,
'Colony': 137,
'Place': 138,
'2360': 139,
'Falls': 140,
'Quincy': 141,
'2169': 142,
'36': 143,
'Paramount': 144,
'Raynham': 145,
'2767': 146,
'450': 147,
'Highland': 148,
'Salem': 149,
'1970': 150,
'1180': 151,
'Seekonk': 152,
'2771': 153,
'1105': 154,
```

```
'Boston': 155,
'Springfield': 156,
'1119': 157,
'100': 158,
'Charlton': 159,
'Sturbridge': 160,
'1566': 161,
'262': 162,
'Swansea': 163,
'Mall': 164,
'2777': 165,
'333': 166,
'Tewksbury': 167,
'1876': 168,
'550': 169,
'Providence': 170,
'Walpole': 171,
'2081': 172,
'352': 173,
'Palmer': 174,
'Ware': 175,
'1082': 176,
'3005': 177,
'Cranberry': 178,
'Rt': 179,
'6': 180,
'28': 181,
'Wareham': 182,
'2538': 183,
'59': 184,
'Airmont': 185,
'NY': 186,
'10901': 187,
'141': 188,
'Extension': 189,
'Albany': 190,
'12205': 191,
'13858': 192,
'31': 193,
'W': 194,
'Albion': 195,
'14411': 196,
'2055': 197,
'Niagara': 198,
'Amherst': 199,
'14228': 200,
'101': 201,
```

```
'Sanford': 202,
'Farm': 203,
'Shpg': 204,
'Center': 205,
'Amsterdam': 206,
'12010': 207,
'297': 208,
'Grant': 209,
'Auburn': 210,
'13021': 211,
'4133': 212,
'Veterans': 213,
'Batavia': 214,
'14020': 215,
'6265': 216,
'Brockport': 217,
'Spencerport': 218,
'14420': 219,
'5399': 220,
'Genesse': 221,
'Camillus': 222,
'13031': 223,
'3191': 224,
'County': 225,
'rd': 226,
'10': 227,
'Canandaigua': 228,
'14424': 229,
'Catskill': 230,
'12414': 231,
'161': 232,
'Centereach': 233,
'11720': 234,
'3018': 235,
'Central': 236,
'Square': 237,
'13036': 238,
'Thruway': 239,
'Plaza': 240,
'Cheektowaga': 241,
'14225': 242,
'8064': 243,
'Brewerton': 244,
'Cicero': 245,
'13039': 246,
'5033': 247,
'Transit': 248,
```

```
'Clarence': 249,
'14031': 250,
'3949': 251,
'Route': 252,
'Clay': 253,
'13041': 254,
'139': 255,
'Merchant': 256,
'Cobleskill': 257,
'12043': 258,
'85': 259,
'Crooked': 260,
'Hill': 261,
'Commack': 262,
'11725': 263,
'872': 264,
'13': 265,
'Cortlandville': 266,
'13045': 267,
'279': 268,
'Troy': 269,
'Greenbush': 270,
'12061': 271,
'2465': 272,
'Hempstead': 273,
'Turnpike': 274,
'Meadow': 275,
'11554': 276,
'6438': 277,
'Basile': 278,
'Rowe': 279,
'Syracuse': 280,
'13057': 281,
'25737': 282,
'US': 283,
'Evans': 284,
'Mills': 285,
'13637': 286,
'901': 287,
'110': 288,
'Farmingdale': 289,
'11735': 290,
'2400': 291,
'9': 292,
'Fishkill': 293,
'12524': 294,
'10401': 295,
```

```
'Bennett': 296,
'Fredonia': 297,
'14063': 298,
'1818': 299,
'3': 300,
'Fulton': 301,
'13069': 302,
'4300': 303,
'Lakeville': 304,
'Geneseo': 305,
'14454': 306,
'990': 307,
'5': 308,
'Geneva': 309,
'14456': 310,
'311': 311,
'RT': 312,
'9W': 313,
'Glenmont': 314,
'12077': 315,
'Dutch': 316,
'Meadows': 317,
'Ln': 318,
'Glenville': 319,
'12302': 320,
'Elm': 321,
'Ridge': 322,
'Greece': 323,
'14626': 324,
'1549': 325,
'Halfmoon': 326,
'12065': 327,
'5360': 328,
'Southwestern': 329,
'Hamburg': 330,
'14075': 331,
'103': 332,
'Caroline': 333,
'Herkimer': 334,
'13350': 335,
'1000': 336,
'Hornell': 337,
'14843': 338,
'1400': 339,
'64': 340,
'Horseheads': 341,
'14845': 342,
```

```
'135': 343,
'Fairgrounds': 344,
'Pkwy': 345,
'Ithaca': 346,
'14850': 347,
'2': 348,
'Gannett': 349,
'Johnson': 350,
'City': 351,
'13790': 352,
'233': 353,
'5th': 354,
'Ext': 355,
'Johnstown': 356,
'12095': 357,
'601': 358,
'Frank': 359,
'Stottile': 360,
'Kingston': 361,
'12401': 362,
'350': 363,
'E': 364,
'Fairmount': 365,
'Lakewood': 366,
'14750': 367,
'4975': 368,
'Lancaster': 369,
'14086': 370,
'579': 371,
'Troy-Schenectady': 372,
'Latham': 373,
'12110': 374,
'5783': 375,
'So': 376,
'Lockport': 377,
'14094': 378,
'7155': 379,
'Lowville': 380,
'13367': 381,
'425': 382,
'Macedon': 383,
'14502': 384,
'3222': 385,
'Malone': 386,
'12953': 387,
'Sunrise': 388,
'Massapequa': 389,
```

```
'11758': 390,
'43': 391,
'Stephenville': 392,
'Massena': 393,
'13662': 394,
'750': 395,
'Middle': 396,
'Country': 397,
'Island': 398,
'11953': 399,
'470': 400,
'211': 401,
'Middletown': 402,
'10940': 403,
'3133': 404,
'Mohegan': 405,
'Lake': 406,
'10547': 407,
'288': 408,
'Larkin': 409,
'Monroe': 410,
'10950': 411,
'41': 412,
'Anawana': 413,
'Monticello': 414,
'12701': 415,
'4765': 416,
'Commercial': 417,
'New': 418,
'13413': 419,
'Newburgh': 420,
'12550': 421,
'255': 422,
'CT': 423,
'6001': 424,
'120': 425,
'Parkway': 426,
'Branford': 427,
'6405': 428,
'Farmington': 429,
'Bristol': 430,
'6010': 431,
'Berlin': 432,
'Cromwell': 433,
'6416': 434,
'67': 435,
'Newton': 436,
```

```
'Danbury': 437,
'6810': 438,
'656': 439,
'Haven': 440,
'Derby': 441,
'6418': 442,
'69': 443,
'Prospect': 444,
'Windsor': 445,
'6088': 446,
'150': 447,
'Gold': 448,
'Star': 449,
'Groton': 450,
'6340': 451,
'900': 452,
'Post': 453,
'Guilford': 454,
'6437': 455,
'2300': 456,
'Dixwell': 457,
'Hamden': 458,
'6514': 459,
'495': 460,
'Flatbush': 461,
'6106': 462,
'Lisbon': 463,
'6351': 464,
'420': 465,
'Buckland': 466,
'Hills': 467,
'Manchester': 468,
'6040': 469,
'1365': 470,
'Milford': 471,
'6460': 472,
'1100': 473,
'Naugatuck': 474,
'6770': 475,
'315': 476,
'Foxon': 477,
'6513': 478,
'164': 479,
'6776': 480,
'3164': 481,
'Newington': 482,
'6111': 483,
```

```
'474': 484,
'Windham': 485,
'6256': 486,
'650': 487,
'Norwalk': 488,
'6851': 489,
'680': 490,
'Connecticut': 491,
'6854': 492,
'220': 493,
'Norwich': 494,
'6360': 495,
'655': 496,
'Old': 497,
'Saybrook': 498,
'6475': 499,
'625': 500,
'School': 501,
'Putnam': 502,
'6260': 503,
'80': 504,
'Town': 505,
'Line': 506,
'Rocky': 507,
'6067': 508,
'465': 509,
'Bridgeport': 510,
'Shelton': 511,
'6484': 512,
'235': 513,
'Queen': 514,
'Southington': 515,
'6489': 516,
'Barnum': 517,
'Cutoff': 518,
'Stratford': 519,
'6614': 520,
'970': 521,
'Torringford': 522,
'Torrington': 523,
'6790': 524,
'844': 525,
'No': 526,
'Wallingford': 527,
'6492': 528,
'910': 529,
'Wolcott': 530,
```

```
'Waterbury': 531,
'6705': 532,
'155': 533,
'Waterford': 534,
'6385': 535,
'515': 536,
'Sawmill': 537,
'West': 538,
'6516': 539,
'2473': 540,
'Hackworth': 541,
'Adamsville': 542,
'AL': 543,
'35005': 544,
'630': 545,
'Coonial': 546,
'Promenade': 547,
'Alabaster': 548,
'35007': 549,
'2643': 550,
'Alexander': 551,
'35010': 552,
'540': 553,
'Bypass': 554,
'Andalusia': 555,
'36420': 556,
'5560': 557,
'Mcclellan': 558,
'Anniston': 559,
'36206': 560,
'1450': 561,
'Brindlee': 562,
'Mtn': 563,
'Arab': 564,
'35016': 565,
'1011': 566,
'Athens': 567,
'35611': 568,
'973': 569,
'Gilbert': 570,
'Ferry': 571,
'Se': 572,
'Attalla': 573,
'35954': 574,
'1717': 575,
'South': 576,
'College': 577,
```

```
'36830': 578,
'701': 579,
'Mcmeans': 580,
'Bay': 581,
'Minette': 582,
'36507': 583,
'Academy': 584,
'Bessemer': 585,
'35022': 586,
'312': 587,
'Palisades': 588,
'Birmingham': 589,
'35209': 590,
'1600': 591,
'Montclair': 592,
'35210': 593,
'5919': 594,
'Trussville': 595,
'Crossings': 596,
'35235': 597,
'9248': 598,
'35206': 599,
'1972': 600,
'431': 601,
'Boaz': 602,
'35957': 603,
'10675': 604,
'Brent': 605,
'35034': 606,
'2041': 607,
'Douglas': 608,
'Brewton': 609,
'36426': 610,
'5100': 611,
'Calera': 612,
'35040': 613,
'1916': 614,
'Point': 615,
'35215': 616,
'1950': 617,
'Centre': 618,
'35960': 619,
'16077': 620,
'Highway': 621,
'Chelsea': 622,
'35043': 623,
'1415': 624,
```

```
'7Th': 625,
'Clanton': 626,
'35045': 627,
'626': 628,
'Olive': 629,
'Sw': 630,
'Cullman': 631,
'35055': 632,
'27520': 633,
'98': 634,
'Daphne': 635,
'36526': 636,
'2800': 637,
'Spring': 638,
'Avn': 639,
'SW': 640,
'Decatur': 641,
'35603': 642,
'969': 643,
'Us': 644,
'Demopolis': 645,
'36732': 646,
'3300': 647,
'Oates': 648,
'Dothan': 649,
'36301': 650,
'4310': 651,
'Montgomery': 652,
'36303': 653,
'600': 654,
'Boll': 655,
'Weevil': 656,
'Circle': 657,
'Enterprise': 658,
'36330': 659,
'3176': 660,
'Eufaula': 661,
'36027': 662,
'7100': 663,
'Aaron': 664,
'Aronov': 665,
'Fairfield': 666,
'35064': 667,
'10040': 668,
'48': 669,
'Fairhope': 670,
'36533': 671,
```

```
'3186': 672,
'171': 673,
'Fayette': 674,
'35555': 675,
'3100': 676,
'Hough': 677,
'Florence': 678,
'35630': 679,
'2200': 680,
'Mckenzie': 681,
'Foley': 682,
'36535': 683,
'2001': 684,
'Glenn': 685,
'Bldv': 686,
'Fort': 687,
'Payne': 688,
'35968': 689,
'340': 690,
'Meighan': 691,
'Gadsden': 692,
'35903': 693,
'890': 694,
'Odum': 695,
'Gardendale': 696,
'35071': 697,
'1608': 698,
'Magnolia': 699,
'36340': 700,
'501': 701,
'Willow': 702,
'Lane': 703,
'Greenville': 704,
'36037': 705,
'170': 706,
'Morgan': 707,
'Gulf': 708,
'Shores': 709,
'36542': 710,
'11697': 711,
'Guntersville': 712,
'35976': 713,
'42417': 714,
'195': 715,
'Haleyville': 716,
'35565': 717,
'1706': 718,
```

```
'Military': 719,
'Hamilton': 720,
'35570': 721,
'NW': 722,
'Hartselle': 723,
'35640': 724,
'209': 725,
'Lakeshore': 726,
'Homewood': 727,
'2780': 728,
'John': 729,
'Hawkins': 730,
'Hoover': 731,
'35244': 732,
'5335': 733,
'35242': 734,
'1007': 735,
'Red': 736,
'Farmer': 737,
'Hueytown': 738,
'35023': 739,
'2900': 740,
'Mem': 741,
'PkwyDrake': 742,
'Huntsville': 743,
'35801': 744,
'11610': 745,
'35803': 746,
'Sparkman': 747,
'35810': 748,
'330': 749,
'Sutton': 750,
'35763': 751,
'6140A': 752,
'Univ': 753,
'35806': 754,
'4206': 755,
'N': 756,
'Jackson': 757,
'36545': 758,
'1625': 759,
'Pelham': 760,
'Jacksonville': 761,
'36265': 762,
'1801': 763,
'78': 764,
'Jasper': 765,
```

```
'35501': 766,
'8551': 767,
'Whitfield': 768,
'Leeds': 769,
'35094': 770,
'8650': 771,
'Madison': 772,
'35758': 773,
'145': 774,
'Kelley': 775,
'Millbrook': 776,
'36054': 777,
'University': 778,
'Mobile': 779,
'36609': 780,
'6350': 781,
'Cottage': 782,
'Beltline': 783,
'36606': 784,
'2500': 785,
'Dawes': 786,
'36695': 787,
'5245': 788,
'Rangeline': 789,
'Service': 790,
'36619': 791,
'685': 792,
'Schillinger': 793,
'3371': 794,
'Alabama': 795,
'Monroeville': 796,
'36460': 797,
'10710': 798,
'Chantilly': 799,
'36117': 800,
'3801': 801,
'Eastern': 802,
'36116': 803,
'6495': 804,
'Atlanta': 805,
'851': 806,
'Ann': 807,
'36107': 808,
'15445': 809,
'24': 810,
'Moulton': 811,
'35650': 812,
```

```
'517': 813,
'Avalon': 814,
'Muscle': 815,
'Shoals': 816,
'35661': 817,
'5710': 818,
'Mcfarland': 819,
'Northport': 820,
'35476': 821,
'2453': 822,
'2Nd': 823,
'Oneonta': 824,
'35121': 825,
'Pepperrell': 826,
'Opelika': 827,
'36801': 828,
'92': 829,
'36203': 830,
'231': 831,
'Ozark': 832,
'36360': 833,
'2181': 834,
'35124': 835,
'165': 836,
'Vaughan': 837,
'Pell': 838,
'35125': 839,
'3700': 840,
'280-431': 841,
'Phenix': 842,
'36867': 843,
'1903': 844,
'Cobbs': 845,
'Ford': 846,
'Prattville': 847,
'36066': 848,
'4180': 849,
'Roanoke': 850,
'36274': 851,
'13675': 852,
'Russellville': 853,
'35653': 854,
'1095': 855,
'Industrial': 856,
'Saraland': 857,
'36571': 858,
'24833': 859,
```

```
'Johnt': 860,
'Reidprkw': 861,
'Scottsboro': 862,
'35768': 863,
'1501': 864,
'14': 865,
'Selma': 866,
'36703': 867,
'7855': 868,
'Moffett': 869,
'Semmes': 870,
'36575': 871,
'Springville': 872,
'Station': 873,
'35146': 874,
'690': 875,
'Sumiton': 876,
'35148': 877,
'41301': 878,
'Sylacauga': 879,
'35150': 880,
'214': 881,
'Haynes': 882,
'Talladega': 883,
'35160': 884,
'1300': 885,
'Gilmer': 886,
'Tallassee': 887,
'36078': 888,
'34301': 889,
'Thomasville': 890,
'36784': 891,
'1420': 892,
'36081': 893,
'Skyland': 894,
'Tuscaloosa': 895,
'35405': 896,
'3501': 897,
'20th': 898,
'Av': 899,
'36854': 900,
'Vestavia': 901,
'35216': 902,
'4538': 903,
'Wetumpka': 904,
'36092': 905,
'2575': 906,
```

```
  'Winfield': 907,
  '35594': 908},
 {'<PAD>': 0, 'address': 1, 'city': 2, 'state': 3, 'zip': 4})
```

[8]:
```
print(f"Task {task_id} - AG tests")

stu_vocab, _ = task_2_solution()

print(f"Task {task_id} - your vocab (first 10 entries):\n{list(stu_vocab.
 ↪items())[:10]}")

assert isinstance(
    stu_vocab, dict
), f"Task {task_id}: The returned `vocab` should be a dictionary."
```

```
Task 1 - AG tests
Task 1 - your vocab (first 10 entries):
[('<PAD>', 0), ('<UNK>', 1), ('777', 2), ('Brockton', 3), ('Avenue', 4),
('Abington', 5), ('MA', 6), ('2351', 7), ('30', 8), ('Memorial', 9)]
```

4 Task 3: Convert Sequences to Indices and Find Longest Sequence

Now, use the maps we created in task 2 to convert the sequences and labels datasets we created in task 1 from datasets of strings to being their corresponding integer representations in the map.

For example, if you have

`[["123", "Main", "St", "Springfield", "IL", "62704"], ["124", "Main", "St", "Springfield", "IL", "62705"], etc.]`

and a vocab of:

```
{     "<PAD>": 0,     "<UNK>": 1,     "123": 2,     "Main": 3,     "St": 4,
"Springfield": 5,     "IL": 6,     "62704": 7,     "124" : 8,     "62705" : 9,
etc. }
```

X should be converted to:

`X = [[2, 3, 4, 5, 6, 7], [8, 3, 4, 5, 6, 9], etc.]`

Do the same for `labels` using `state_to_idx` to create `y`

Now is also a good time to find the longest sequence of tokens in your sequences array. For example, the example entry we have above in X is 6 tokens long, and if this was the longest sequence in our dataset, we'd return 6 as an integer. Find the longest sequence and return it in the integer `max_length`.

[9]:
```
task_id = '3'

## Utility Code will go here
```

[14]:
```
def task_3_solution():
    sequences, labels = task_1_solution()
    vocab, state_to_idx = task_2_solution()

    X = []
    y = []
    max_length = 0

    # YOUR CODE HERE
    #raise NotImplementedError()
    for s in sequences:
        x = []
        for w in s:
            x.append(vocab.get(w, UNK_TOKEN))
        X.append(x)
        max_length = max(max_length, len(s))
    for label in labels:
        l = []
        for w in label:
            l.append(state_to_idx[w])
        y.append(l)

    return X, y, max_length
```

[15]:
```
# Use this cell to explore your solution.

task_3_solution()
```

[15]:
```
([[2, 3, 4, 5, 6, 7],
  [8, 9, 10, 11, 6, 12],
  [13, 14, 4, 15, 6, 16],
  [17, 18, 19, 3, 6, 20],
  [21, 22, 23, 24, 6, 25],
  [26, 9, 27, 28, 6, 29],
  [30, 31, 32, 33, 34, 6, 35],
  [36, 37, 38, 39, 40, 6, 41],
  [42, 43, 44, 33, 43, 6, 45],
  [46, 47, 48, 49, 50, 51, 52, 6, 53],
  [54, 55, 23, 56, 6, 57],
  [58, 59, 50, 60, 6, 61],
  [62, 63, 64, 65, 6, 66],
  [67, 68, 19, 69, 6, 70],
  [71, 72, 64, 73, 6, 74],
  [75, 72, 19, 76, 6, 77],
```

```
[78, 79, 27, 80, 6, 81],
[82, 83, 84, 85, 6, 86],
[87, 88, 89, 90, 6, 91],
[92, 93, 94, 6, 95],
[96, 97, 98, 19, 99, 6, 100],
[101, 102, 9, 38, 103, 104, 6, 105],
[106, 48, 72, 64, 103, 107, 6, 108],
[109, 110, 84, 103, 111, 6, 112],
[113, 114, 19, 103, 115, 6, 116],
[117, 114, 64, 103, 118, 6, 119],
[120, 121, 19, 122, 6, 123],
[124, 103, 125, 19, 126, 6, 127],
[128, 39, 114, 64, 129, 6, 130],
[128, 131, 132, 133, 134, 6, 135],
[136, 137, 138, 68, 6, 139],
[87, 140, 50, 141, 6, 142],
[143, 144, 10, 145, 6, 146],
[147, 148, 89, 149, 6, 150],
[151, 51, 52, 4, 152, 6, 153],
[154, 155, 84, 156, 6, 157],
[158, 159, 84, 160, 6, 161],
[162, 163, 164, 27, 163, 6, 165],
[166, 114, 19, 167, 6, 168],
[169, 170, 38, 171, 6, 172],
[173, 174, 84, 175, 6, 176],
[177, 178, 38, 179, 180, 181, 182, 6, 183],
[13, 179, 184, 185, 186, 187],
[188, 72, 89, 189, 190, 186, 191],
[192, 179, 193, 194, 195, 186, 196],
[197, 198, 140, 50, 199, 186, 200],
[201, 202, 203, 204, 205, 206, 186, 207],
[208, 209, 4, 210, 186, 211],
[212, 213, 9, 10, 214, 186, 215],
[216, 217, 218, 23, 217, 186, 219],
[220, 194, 221, 64, 222, 186, 223],
[224, 225, 226, 227, 228, 186, 229],
[8, 230, 230, 186, 231],
[232, 233, 164, 233, 186, 234],
[235, 39, 89, 236, 237, 186, 238],
[158, 239, 240, 241, 186, 242],
[243, 244, 23, 245, 186, 246],
[247, 248, 84, 249, 186, 250],
[251, 252, 193, 253, 186, 254],
[255, 256, 138, 257, 186, 258],
[259, 260, 261, 84, 262, 186, 263],
[264, 252, 265, 266, 186, 267],
[268, 269, 84, 39, 270, 186, 271],
```

```
[272, 273, 274, 39, 275, 186, 276],
[277, 278, 279, 39, 280, 186, 281],
[282, 283, 179, 82, 284, 285, 186, 286],
[287, 252, 288, 289, 186, 290],
[291, 252, 292, 293, 186, 294],
[295, 296, 84, 297, 186, 298],
[299, 110, 252, 300, 301, 186, 302],
[303, 304, 84, 305, 186, 306],
[307, 252, 308, 78, 309, 186, 310],
[311, 312, 313, 314, 186, 315],
[120, 316, 317, 318, 319, 186, 320],
[158, 321, 322, 205, 27, 323, 186, 324],
[325, 179, 292, 326, 186, 327],
[328, 329, 50, 330, 186, 331],
[332, 103, 333, 64, 334, 186, 335],
[336, 110, 252, 143, 337, 186, 338],
[339, 225, 23, 340, 341, 186, 342],
[343, 344, 9, 345, 346, 186, 347],
[348, 349, 27, 350, 351, 186, 352],
[353, 354, 89, 355, 356, 186, 357],
[358, 359, 360, 50, 361, 186, 362],
[363, 364, 365, 89, 366, 186, 367],
[368, 248, 23, 369, 186, 370],
[371, 372, 84, 373, 186, 374],
[375, 376, 248, 84, 377, 186, 378],
[379, 110, 179, 133, 48, 380, 186, 381],
[382, 252, 193, 383, 186, 384],
[385, 110, 179, 82, 386, 186, 387],
[120, 388, 164, 389, 186, 390],
[391, 392, 64, 393, 186, 394],
[395, 396, 397, 84, 396, 398, 186, 399],
[400, 252, 401, 39, 402, 186, 403],
[404, 364, 114, 64, 405, 406, 186, 407],
[408, 409, 410, 186, 411],
[412, 413, 406, 84, 414, 186, 415],
[416, 417, 10, 418, 14, 186, 419],
[135, 179, 136, 420, 186, 421],
[422, 194, 114, 64, 11, 423, 424],
[425, 417, 426, 427, 423, 428],
[339, 429, 89, 430, 423, 431],
[232, 432, 84, 433, 423, 434],
[435, 436, 23, 437, 423, 438],
[439, 418, 440, 89, 441, 423, 442],
[443, 444, 261, 84, 39, 445, 423, 446],
[447, 448, 449, 38, 450, 423, 451],
[452, 155, 453, 84, 454, 423, 455],
[456, 457, 89, 458, 423, 459],
```

```
[460, 461, 89, 14, 423, 462],
[124, 52, 23, 463, 423, 464],
[465, 466, 467, 27, 468, 423, 469],
[470, 155, 453, 84, 471, 423, 472],
[473, 418, 440, 84, 474, 423, 475],
[476, 477, 50, 418, 440, 423, 478],
[479, 437, 23, 418, 471, 423, 480],
[481, 432, 274, 482, 423, 483],
[484, 155, 453, 84, 103, 485, 423, 486],
[487, 114, 89, 488, 423, 489],
[490, 491, 4, 488, 423, 492],
[493, 149, 274, 494, 423, 495],
[496, 155, 453, 23, 497, 498, 423, 499],
[500, 501, 19, 502, 423, 503],
[504, 505, 506, 23, 507, 261, 423, 508],
[509, 510, 4, 511, 423, 512],
[513, 514, 64, 515, 423, 516],
[447, 517, 4, 518, 519, 423, 520],
[521, 522, 19, 523, 423, 524],
[525, 526, 137, 84, 527, 423, 528],
[529, 530, 64, 531, 423, 532],
[533, 534, 426, 526, 534, 423, 535],
[536, 537, 84, 538, 440, 423, 539],
[540, 541, 84, 542, 543, 544],
[545, 546, 547, 345, 548, 543, 549],
[550, 38, 75, 538, 551, 351, 543, 552],
[553, 538, 554, 555, 543, 556],
[557, 558, 50, 559, 543, 560],
[561, 526, 562, 563, 345, 564, 543, 565],
[566, 283, 38, 117, 39, 567, 543, 568],
[569, 570, 571, 84, 572, 573, 543, 574],
[575, 576, 577, 19, 210, 543, 578],
[579, 580, 89, 581, 582, 543, 583],
[395, 584, 10, 585, 543, 586],
[587, 588, 50, 589, 543, 590],
[591, 592, 23, 589, 543, 593],
[594, 595, 596, 345, 589, 543, 597],
[598, 426, 39, 589, 543, 599],
[600, 38, 601, 602, 543, 603],
[604, 38, 308, 605, 543, 606],
[607, 608, 4, 609, 543, 610],
[611, 38, 193, 612, 543, 613],
[614, 205, 615, 23, 205, 615, 543, 616],
[617, 194, 114, 64, 618, 543, 619],
[620, 621, 75, 622, 543, 623],
[624, 625, 19, 576, 626, 543, 627],
[628, 629, 19, 630, 631, 543, 632],
```

```
[633, 38, 634, 635, 543, 636],
[637, 638, 639, 640, 641, 543, 642],
[643, 644, 38, 504, 538, 645, 543, 646],
[647, 576, 648, 19, 649, 543, 650],
[651, 652, 38, 649, 543, 653],
[654, 655, 656, 657, 658, 543, 659],
[660, 576, 661, 4, 661, 543, 662],
[663, 664, 665, 10, 666, 543, 667],
[668, 225, 84, 669, 670, 543, 671],
[672, 38, 673, 103, 674, 543, 675],
[676, 677, 23, 678, 543, 679],
[680, 576, 681, 64, 682, 543, 683],
[684, 685, 686, 630, 687, 688, 543, 689],
[690, 39, 691, 50, 692, 543, 693],
[694, 695, 84, 696, 543, 697],
[698, 194, 699, 89, 309, 543, 700],
[701, 702, 703, 704, 543, 705],
[706, 687, 707, 84, 708, 709, 543, 710],
[711, 283, 38, 601, 712, 543, 713],
[714, 38, 715, 716, 543, 717],
[718, 719, 19, 576, 720, 543, 721],
[135, 38, 193, 722, 723, 543, 724],
[725, 726, 426, 727, 543, 590],
[728, 729, 730, 345, 731, 543, 732],
[733, 38, 75, 576, 731, 543, 734],
[735, 736, 737, 10, 738, 543, 739],
[740, 48, 741, 742, 89, 743, 543, 744],
[745, 9, 345, 576, 743, 543, 746],
[680, 747, 10, 743, 543, 748],
[749, 750, 23, 743, 543, 751],
[752, 753, 10, 743, 543, 754],
[755, 756, 577, 89, 757, 543, 758],
[759, 760, 576, 761, 543, 762],
[763, 38, 764, 39, 765, 543, 766],
[767, 768, 89, 769, 543, 770],
[771, 772, 50, 772, 543, 773],
[774, 775, 50, 776, 543, 777],
[150, 48, 778, 50, 779, 543, 780],
[781, 782, 261, 84, 779, 543, 780],
[201, 576, 783, 621, 779, 543, 784],
[785, 786, 84, 779, 543, 787],
[788, 789, 790, 23, 779, 543, 791],
[792, 793, 23, 779, 543, 787],
[794, 48, 795, 89, 796, 543, 797],
[798, 799, 345, 652, 543, 800],
[801, 802, 50, 652, 543, 803],
[804, 805, 38, 652, 543, 800],
```

```
 [806, 807, 64, 652, 543, 808],
 [809, 621, 810, 811, 543, 812],
 [813, 538, 814, 89, 815, 816, 543, 817],
 [818, 819, 50, 820, 543, 821],
 [822, 823, 4, 39, 824, 543, 825],
 [740, 826, 345, 827, 543, 828],
 [829, 240, 703, 115, 543, 830],
 [116, 38, 831, 576, 832, 543, 833],
 [834, 760, 345, 760, 543, 835],
 [836, 837, 318, 838, 351, 543, 839],
 [840, 38, 841, 756, 842, 351, 543, 843],
 [844, 845, 846, 23, 847, 543, 848],
 [849, 644, 38, 601, 850, 543, 851],
 [852, 38, 391, 853, 543, 854],
 [855, 856, 345, 857, 543, 858],
 [859, 860, 861, 862, 543, 863],
 [864, 38, 865, 39, 866, 543, 867],
 [868, 869, 23, 870, 543, 871],
 [447, 872, 873, 50, 872, 543, 874],
 [875, 38, 764, 876, 543, 877],
 [878, 283, 38, 75, 879, 543, 880],
 [881, 882, 19, 883, 543, 884],
 [885, 886, 89, 887, 543, 888],
 [889, 38, 391, 890, 543, 891],
 [892, 644, 831, 576, 269, 543, 893],
 [864, 894, 50, 364, 895, 543, 896],
 [897, 898, 899, 98, 543, 900],
 [885, 652, 621, 901, 467, 543, 902],
 [903, 644, 38, 831, 904, 543, 905],
 [906, 644, 38, 391, 907, 543, 908]],
[[1, 1, 1, 2, 3, 4],
 [1, 1, 1, 2, 3, 4],
 [1, 1, 1, 2, 3, 4],
 [1, 1, 1, 2, 3, 4],
 [1, 1, 1, 2, 3, 4],
 [1, 1, 1, 2, 3, 4],
 [1, 1, 1, 1, 2, 3, 4],
 [1, 1, 1, 2, 2, 3, 4],
 [1, 1, 1, 1, 2, 3, 4],
 [1, 1, 1, 1, 1, 2, 2, 3, 4],
 [1, 1, 1, 2, 3, 4],
 [1, 1, 1, 2, 3, 4],
 [1, 1, 1, 2, 3, 4],
 [1, 1, 1, 2, 3, 4],
 [1, 1, 1, 2, 3, 4],
 [1, 1, 1, 2, 3, 4],
 [1, 1, 1, 2, 3, 4],
```

```
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 2, 3, 4],
[1, 1, 1, 1, 2, 2, 3, 4],
[1, 1, 1, 2, 2, 3, 4],
[1, 1, 1, 2, 2, 3, 4],
[1, 1, 1, 2, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 2, 3, 4],
[1, 1, 1, 2, 2, 3, 4],
```

```
[1, 1, 1, 2, 2, 3, 4],
[1, 1, 1, 1, 2, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 2, 3, 4],
[1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
```

```
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 2, 3, 4],
[1, 1, 1, 2, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
```

```
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
[1, 1, 1, 2, 3, 4],
```

```
  [1, 1, 1, 2, 3, 4],
  [1, 1, 1, 1, 2, 2, 3, 4],
  [1, 1, 1, 2, 3, 4],
  [1, 1, 1, 1, 2, 3, 4],
  [1, 1, 1, 2, 3, 4],
  [1, 1, 1, 2, 3, 4],
  [1, 1, 1, 1, 2, 3, 4],
  [1, 1, 1, 2, 3, 4],
  [1, 1, 1, 2, 2, 3, 4],
  [1, 1, 1, 1, 2, 2, 3, 4],
  [1, 1, 1, 1, 2, 3, 4],
  [1, 1, 1, 1, 2, 3, 4],
  [1, 1, 1, 2, 3, 4],
  [1, 1, 1, 2, 3, 4],
  [1, 1, 1, 2, 3, 4],
  [1, 1, 1, 1, 2, 3, 4],
  [1, 1, 1, 2, 3, 4],
  [1, 1, 1, 1, 2, 3, 4],
  [1, 1, 1, 2, 3, 4],
  [1, 1, 1, 1, 2, 3, 4],
  [1, 1, 1, 2, 3, 4],
  [1, 1, 1, 2, 3, 4],
  [1, 1, 1, 2, 3, 4],
  [1, 1, 1, 1, 2, 3, 4],
  [1, 1, 1, 1, 2, 3, 4],
  [1, 1, 1, 2, 3, 4],
  [1, 1, 1, 2, 2, 3, 4],
  [1, 1, 1, 1, 2, 3, 4],
  [1, 1, 1, 1, 2, 3, 4]],
 9)
```

[16]:
```python
print(f"Task {task_id} - AG tests")

stu_X, stu_y, stu_max_length = task_3_solution()

print(f"Task {task_id} - your tokenized sequences (first 2):\n{stu_X[:2]}")
print(f"Task {task_id} - your label indices (first 2):\n{stu_y[:2]}")
print(f"Task {task_id} - your max sequence length: {stu_max_length}")

assert isinstance(
    stu_X, list
), f"Task {task_id}: The returned `X` should be a list of tokenized sequences."
assert isinstance(
    stu_y, list
), f"Task {task_id}: The returned `y` should be a list of label index sequences.
 ↪"
assert isinstance(
```

```
    stu_max_length, int
), f"Task {task_id}: The returned `max_length` should be an integer."

assert len(stu_X) == len(
    stu_y
), f"Task {task_id}: The number of tokenized sequences in `X` does not match␣
 ↪the number of label sequences in `y`."

for seq in stu_X:
    assert isinstance(seq, list), f"Task {task_id}: Each sequence in `X` should␣
 ↪be a list."
    assert all(
        isinstance(token, int) for token in seq
    ), f"Task {task_id}: Each token in `X` should be an integer."
for seq in stu_y:
    assert isinstance(seq, list), f"Task {task_id}: Each sequence in `y` should␣
 ↪be a list."
    assert all(
        isinstance(label, int) for label in seq
    ), f"Task {task_id}: Each label in `y` should be an integer."

assert all(
    len(seq) <= stu_max_length for seq in stu_X
), f"Task {task_id}: All sequences in `X` should have lengths less than or␣
 ↪equal to `max_length`."
assert all(
    len(seq) <= stu_max_length for seq in stu_y
), f"Task {task_id}: All sequences in `y` should have lengths less than or␣
 ↪equal to `max_length`."
```

```
Task 4 - AG tests
Task 4 - your tokenized sequences (first 2):
[[2, 3, 4, 5, 6, 7], [8, 9, 10, 11, 6, 12]]
Task 4 - your label indices (first 2):
[[1, 1, 1, 2, 3, 4], [1, 1, 1, 2, 3, 4]]
Task 4 - your max sequence length: 9
```

5 Task 4: Create Dataset class

Now, we need to wrap the data we processed in the previous tasks into dataset objects.

This function will return two instances of these objects, being a train and test dataset. The return statement has been defined for you.

Your job here is to complete the `__init__` function, which will assign padded versions of X and y to `self.X` and `self.y`. For example:

For X like:

`X = [[2, 3, 4], [5, 6]]`

If `max_length` is 5, we would assign

```
[[2, 3, 4, 0, 0],  # Padded to length 5`
[5, 6, 0, 0, 0]]
```

to `self.X`. Recall that `vocab[PAD_TOKEN] = 0`

Do the same with `self.y`

[13]:
```
task_id = '4'

## Utility Code will go here
```

[18]:
```
def task_4_solution():
    X, y, max_length = task_3_solution()
    vocab, state_to_idx = task_2_solution()

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
 ↪random_state=42)


    class AddressDataset(Dataset):
        def __init__(self, X, y):
            self.X = X
            self.y = y
            # YOUR CODE HERE
            #raise NotImplementedError()
            for i in range(len(self.X)):
                self.X[i] = self.X[i] + [vocab[PAD_TOKEN] for _ in␣
 ↪range(max_length - len(self.X[i]))]
                self.y[i] = self.y[i] + [vocab[PAD_TOKEN] for _ in␣
 ↪range(max_length - len(self.y[i]))]

            # This code should come after your code adding the padding
            self.X = torch.tensor(self.X, dtype=torch.long)
            self.y = torch.tensor(self.y, dtype=torch.long)


        def __len__(self):
            return len(self.X)

        def __getitem__(self, idx):
            return self.X[idx], self.y[idx]

    train_dataset = AddressDataset(X_train, y_train)
    test_dataset = AddressDataset(X_test, y_test)
```

```
    return train_dataset, test_dataset
```

[19]:
```
# Use this cell to explore your solution.

task_4_solution()
```

[19]:
```
(<__main__.task_4_solution.<locals>.AddressDataset at 0x723daed1b3d0>,
 <__main__.task_4_solution.<locals>.AddressDataset at 0x723d16b2d180>)
```

[20]:
```
print(f"Task {task_id} - AG tests")

# Call the student's solution
train_dataset, test_dataset = task_4_solution()

assert isinstance(
    train_dataset, Dataset
), f"Task {task_id}: `train_dataset` should be an instance of `torch.utils.data.
 ↪Dataset`."
assert isinstance(
    test_dataset, Dataset
), f"Task {task_id}: `test_dataset` should be an instance of `torch.utils.data.
 ↪Dataset`."

assert len(train_dataset) > 0, f"Task {task_id}: `train_dataset` should not be␣
 ↪empty."
assert len(test_dataset) > 0, f"Task {task_id}: `test_dataset` should not be␣
 ↪empty."

train_sample_X, train_sample_y = train_dataset[0]  # First training sample
assert isinstance(
    train_sample_X, torch.Tensor
), f"Task {task_id}: Features from `train_dataset` should be a torch.Tensor."
assert isinstance(
    train_sample_y, torch.Tensor
), f"Task {task_id}: Labels from `train_dataset` should be a torch.Tensor."

assert len(train_sample_X) == len(
    train_sample_y
), f"Task {task_id}: The feature and label tensors should have the same length␣
 ↪after padding."
```

```
Task 4 - AG tests
```

6 Task 5: Build Model

In this task, you will configure the BiLSTM-CRF model.

As you can see, we've defined some starting parameters for you.

Your job here is to define the `__init__` and `forward` methods.

Inside of the init function, we will be defining the layers of the model:

- create `self.embedding` with `nn.embedding`, with num_embeddings set as the vocab_size, embedding_dim equal to the given embedding_dim, and padding_idx being provided as the arguments.
- create `self.lstm` with nn.LSTM, with `embedding_dim` and `hidden_dim` as the first two arguments with `batch_first` and `bidirectional` set to `True`.
- create `hidden2label` with nn.Linear, with the `in_features` argument set to twice the value of hidden_dim, and out_features as `num_labels`
- create `self.crf` with `CRF()`, with the `num_tags` argument as `num_labels` and `batch_first` set as True.

We have defined the `forward` function for you, which is used when training the model and making predictions.

[21]:
```
task_id = '5'

## Utility Code will go here
```

[22]:
```
def task_5_solution():
    vocab, state_to_idx = task_2_solution()

    vocab_size = len(vocab)
    embedding_dim = 100
    hidden_dim = 128
    num_labels = len(state_to_idx)
    pad_idx = vocab[PAD_TOKEN]


    class BiLSTMCRF(nn.Module):
        def __init__(self, vocab_size, embedding_dim, hidden_dim, num_labels,␣
↪pad_idx):
            super(BiLSTMCRF, self).__init__()
            # YOUR CODE HERE
            #raise NotImplementedError()
            self.embedding = nn.Embedding(vocab_size, embedding_dim,␣
↪padding_idx=pad_idx)
            self.lstm = nn.LSTM(embedding_dim, hidden_dim, batch_first=True,␣
↪bidirectional=True)
            self.hidden2label = nn.Linear(hidden_dim * 2, num_labels)
            self.crf = CRF(num_labels, batch_first=True)

        def forward(self, x, y=None, mask=None):
            embeds = self.embedding(x)
```

```
            lstm_out, _ = self.lstm(embeds)
            emissions = self.hidden2label(lstm_out)
            if y is not None:  # Training
                return -self.crf(emissions, y, mask=mask)
            else:  # Prediction
                return self.crf.decode(emissions, mask=mask)



    model = BiLSTMCRF(vocab_size, embedding_dim, hidden_dim, num_labels,␣
 ↪pad_idx)


    return model
```

[23]:
```
# Use this cell to explore your solution.

task_5_solution()
```

[23]:
```
BiLSTMCRF(
  (embedding): Embedding(909, 100, padding_idx=0)
  (lstm): LSTM(100, 128, batch_first=True, bidirectional=True)
  (hidden2label): Linear(in_features=256, out_features=5, bias=True)
  (crf): CRF(num_tags=5)
)
```

[24]:
```
print(f"Task {task_id} - AG tests")

stu_ans = task_5_solution()

assert isinstance(
    stu_ans, nn.Module
), f"Task {task_id}: The returned `model` should be an instance of `torch.nn.
 ↪Module`."

assert hasattr(
    stu_ans, "embedding"
), f"Task {task_id}: The model should have an attribute `embedding`."
assert isinstance(
    stu_ans.embedding, nn.Embedding
), f"Task {task_id}: The `embedding` attribute should be an instance of `torch.
 ↪nn.Embedding`."
assert (
    stu_ans.embedding.padding_idx == 0
), f"Task {task_id}: The `embedding` layer should use `PAD_TOKEN` as its␣
 ↪`padding_idx`."
```

```
assert hasattr(
    stu_ans, "lstm"
), f"Task {task_id}: The model should have an attribute `lstm`."
assert isinstance(
    stu_ans.lstm, nn.LSTM
), f"Task {task_id}: The `lstm` attribute should be an instance of `torch.nn.
 ↪LSTM`."
assert stu_ans.lstm.bidirectional, f"Task {task_id}: The `lstm` layer should be␣
 ↪bidirectional."
assert (
    stu_ans.lstm.input_size == 100
), f"Task {task_id}: The `lstm` layer should have an input size equal to␣
 ↪`embedding_dim`."
assert (
    stu_ans.lstm.hidden_size == 128
), f"Task {task_id}: The `lstm` layer should have a hidden size equal to␣
 ↪`hidden_dim`."

assert hasattr(
    stu_ans, "hidden2label"
), f"Task {task_id}: The model should have an attribute `hidden2label`."
assert isinstance(
    stu_ans.hidden2label, nn.Linear
), f"Task {task_id}: The `hidden2label` attribute should be an instance of␣
 ↪`torch.nn.Linear`."
assert (
    stu_ans.hidden2label.in_features == 256
), f"Task {task_id}: The `hidden2label` layer should have input features equal␣
 ↪to `hidden_dim * 2`."

assert hasattr(
    stu_ans, "crf"
), f"Task {task_id}: The model should have an attribute `crf`."
assert isinstance(
    stu_ans.crf, CRF
), f"Task {task_id}: The `crf` attribute should be an instance of `torchcrf.
 ↪CRF`."
```

```
Task 5 - AG tests
```

7 Task 6: Train Model and calculate epoch loss

In this task, you will train the BiLSTM-CRF model you created in the previous task.

Some of the prerequisites have been set for you, but your job will be to implement the loop which trains the model and calculates the loss for each epoch.

You should loop through each epoch value, in our case (0, 1, 2…. 9). Inside of this loop, you will

calculate the epoch loss for each epoch value. Now, for each epoch value:

- Iterate over the X and y batches from the DataLoader we defined for you, `train_loader`. While iterating through each X_batch(batch in X), and y_batch(batch in y)
  - Create a boolean variable that will serve as our mask for this batch which is `true` if X_batch is not equal to model.embedding.padding_idx, and false otherwise.
  - Compute the CRF `loss` object by passing the input (X_batch), true labels (y_batch), and mask to the model (`loss = model(X_batch, ...)`)
  - call `.zero_grad()` on the optimizer object we created for you to zero the gradients.
  - backpropogate the losses by calling `.backward()` on the loss object.
  - call `.step()` on the optimizer to update the parameters.
  - You now have the epoch loss value stored in `loss.item()` This will accumulate as you iterate through train_loader, and you will append the final value to `epoch_losses` each time you leave the train_loader loop to iterate through the next epoch value.
  - make sure to zero the epoch loss after iterating through each epoch value.

[26]:
```
task_id = '6'

## Utility Code will go here
```

[34]:
```
def task_6_solution():
    train_dataset, _ = task_4_solution()
    model = task_5_solution()
    train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=32,␣
 ↪shuffle=False)
    optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
    num_epochs = 10
    epoch_losses = []
    model.train()
    # YOUR CODE HERE
    #raise NotImplementedError()
    for epoch in range(num_epochs):
        epoch_loss = 0.0
        for X_batch, y_batch in train_loader:
            mask = X_batch != model.embedding.padding_idx
            loss = model(X_batch, y_batch, mask=mask)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
            epoch_loss += loss.item()
        epoch_losses.append(epoch_loss)

    return model, epoch_losses
```

[35]:
```
# Use this cell to explore your solution.

task_6_solution()
```

[35]:
```
(BiLSTMCRF(
   (embedding): Embedding(909, 100, padding_idx=0)
   (lstm): LSTM(100, 128, batch_first=True, bidirectional=True)
   (hidden2label): Linear(in_features=256, out_features=5, bias=True)
   (crf): CRF(num_tags=5)
 ),
 [1831.9642944335938,
  1604.2127075195312,
  1310.9455261230469,
  922.1139221191406,
  605.7126159667969,
  354.44820404052734,
  210.78704833984375,
  123.70899963378906,
  77.47201728820801,
  49.71729278564453])
```

[ ]:
```
print(f"Task {task_id} - AG tests")
stu_model, stu_epoch_losses = task_6_solution()

print(f"Task {task_id} - Your epoch losses:\n{stu_epoch_losses}")

assert isinstance(
    stu_model, nn.Module
), f"Task {task_id}: Your function should return a PyTorch model (nn.Module)."


assert isinstance(
    stu_epoch_losses, list
), f"Task {task_id}: Your function should return a list of epoch losses as the␣
 ↪second output."

assert len(
    stu_epoch_losses
) == 10, f"Task {task_id}: Your returned list of epoch losses should have 10␣
 ↪elements (one for each epoch)."

assert all(
    isinstance(loss, float) for loss in stu_epoch_losses
), f"Task {task_id}: All elements in your epoch losses list should be of type␣
 ↪float."

assert all(
```

```
    stu_epoch_losses[i] >= stu_epoch_losses[i + 1]
    for i in range(len(stu_epoch_losses) - 1)
), f"Task {task_id}: Your epoch losses should generally decrease over time.␣
 ↪Make sure you are appending your values to the end of the return array after␣
 ↪each iteration. "
```

8 Task 7: Evaluate Model

In this task, you will evaluate the BiLSTM-CRF model you trained in the previous task.

This is similar to the last task: Some of the prerequisites have been set for you, but your job will be to implement the loop which evaluates the model and calculates it's accuracy score.

In the loop,

- Iterate through `X_batch, y_batch` from the DataLoader we defined for you, `test_loader`.
  - Create a mask for padding tokens just like you did in the previous part.
  - Compute `preds` by passing the input (X_batch) and mask to the model (`preds = model(X_batch, ...)`). This is the same thing you assigned to the `loss` variable in the previous step.
  - You now have the predicted labels for this batch of addresses in preds, and the actual labels in y_batch. in the form of a a list of lists.

  You now have enough information to write code computing the accuracy of the model. Code is given to you which encourages finding the total number of tokens and the total number of correct tokens and calculating accuracy this way, but you may do this any way you wish.

Accuracy should look like this. Let's say you have

`preds = [[1, 1, 2, 3], [1, 1, 2, 3], [1, 1, 2, 3]]`

and

`y_batch =  [[1, 1, 3, 4, 0], [1, 1, 3, 4, 0], [1, 1, 3, 4, 0]]`

Accuracy would be 50%.

REMEMBER: `preds` does not contain padding, while `y_batch` does.

[30]:
```
task_id = '7'

## Utility Code will go here

_, test_dataset = task_4_solution()
model, _ = task_6_solution()  # Call training to retrieve the trained model
test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=32)
model.eval()
```

[30]:
```
BiLSTMCRF(
  (embedding): Embedding(909, 100, padding_idx=0)
```

```
  (lstm): LSTM(100, 128, batch_first=True, bidirectional=True)
  (hidden2label): Linear(in_features=256, out_features=5, bias=True)
  (crf): CRF(num_tags=5)
)
```

[31]:
```
def task_7_solution():


    total_labels = 0
    correct_labels = 0

    for X_batch, y_batch in test_loader:
        y_batch = y_batch.tolist()
        # YOUR CODE HERE
        #raise NotImplementedError()
        mask = X_batch != model.embedding.padding_idx
        preds = model(X_batch, mask=mask)
        for i in range(len(preds)):
            correct_labels += sum(pred == true for pred, true in zip(preds[i],␣
 ↪y_batch[i][:len(preds[i])]))
            total_labels += len(preds[i])

    accuracy = correct_labels / total_labels if total_labels > 0 else 0


    return accuracy
```

[32]:
```
# Use this cell to explore your solution.

task_7_solution()
```

[32]:
```
0.9873817034700315
```

[33]:
```
print(f"Task {task_id} - AG tests")
stu_accuracy = task_7_solution()

print(f"Task {task_id} - Your accuracy: {stu_accuracy:.4f}")

assert isinstance(stu_accuracy, float), f"Task {task_id}: Your accuracy should␣
 ↪be a float."
assert 0 <= stu_accuracy <= 1, f"Task {task_id}: Your accuracy should be␣
 ↪between 0 and 1."
```

```
Task 7 - AG tests
Task 7 - Your accuracy: 0.9874
```

[ ]:
```

```